IE437 Data-Driven Decision Making and Control
2024 Spring Semester

5. Data-Driven Design Optimization (Surrogate Model Based)

Jinkyoo Park
Associate Professor
Industrial and Systems Engineering Department

Introduction

Motivation

Many problems in science and engineering involve optimizing an expensive black-box function over a high-dimensional space.

- Drug Discovery, Materials Design, Hardware Manufacturing, …
- Black-box optimization problem can be formulated as

$$\text{find } \mathbf{x}^* = \underset{\mathbf{x} \in \mathcal{X}}{\operatorname{argmax}} f(\mathbf{x}) \text{ with a given budget of evaluations } T$$

- Typical approaches for solving Black-box optimization problems:

  Genetic Algorithms, Evolutionary Strategies, Bayesian Optimization, Policy Gradient, …

(Online) Black-box Optimization

Gradient Ascent

- If we have an access to gradient information $\nabla_x f(x)$, then we perform gradient ascent to find an optimal point

$$x_{t+1} = x_t - \eta \nabla_{x_t} f(x_t), \qquad \forall t = 1, \cdots, T$$

- Recap) Optimality Criterion of Convex Function

Let the convex problem is $\min_x f(x)$ where $f(x)$ is the differentiable objective function, and D is the constraint set.

x^* is optimal if and only if it is feasible ($x \in D$) and $\nabla_x f(x)^T (y - x) \geq 0, \ \forall y \in D$

(Online) Black-box Optimization

Gradient Ascent

- If we have an access to gradient information $\nabla_x f(x)$, then we perform gradient ascent to find an optimal point

$$x_{t+1} = x_t - \eta \nabla_{x_t} f(x_t), \qquad \forall t = 1, \cdots, T$$

- However, in most cases, we do not have gradient information of the target black-box function
- Instead, we can train a proxy model $f_\theta(x)$ given data so far, which is differentiable, and perform gradient ascent on the proxy model to find an optimal point

[Gradient Ascent for Black-box Optimization]

For $t = 1, \cdots, T-1$ do
  Train $f_\theta(x)$ with the dataset acquired so far $\mathcal{D}_t = \{(x_i, y_i)\}_{i=1}^t$
  Choose $x_{t+1} = x_t - \nabla_x f_\theta(x_t)$ and evaluate $y_{t+1} = f(x_{t+1})$
  Update $\mathcal{D}_{t+1} \leftarrow \mathcal{D}_t \cup \{(x_{t+1}, y_{t+1})\}$

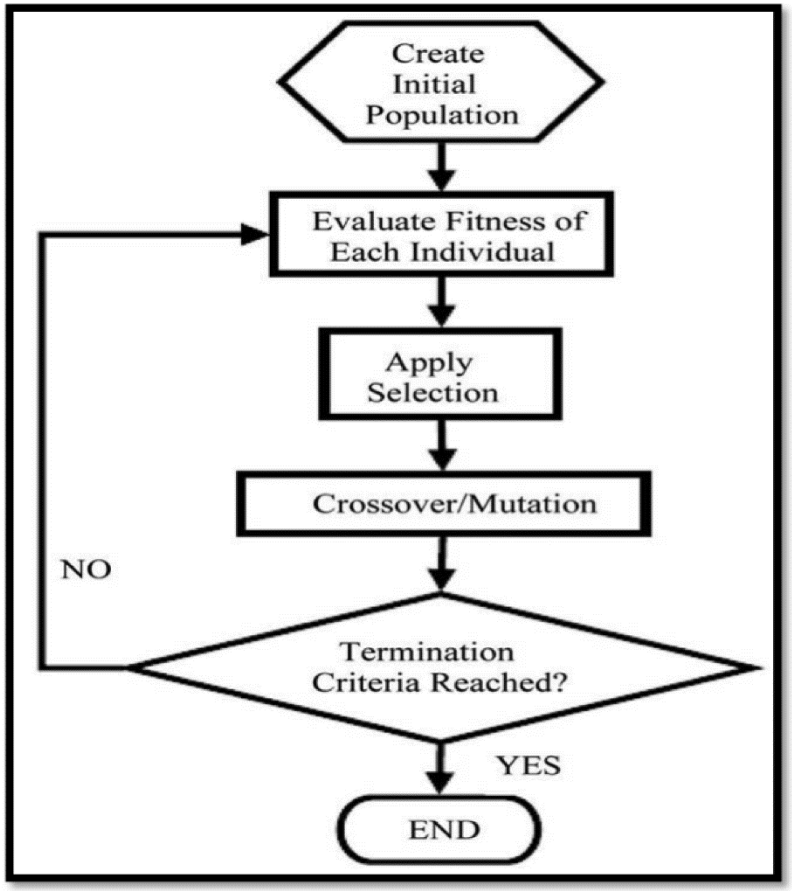
(Online) Black-box Optimization

Genetic Algorithms

- Genetic algorithms are inspired by Darwin's theory of evolution, which explains how species evolve over generations through processes such as selection, crossover, and mutation.

- Procedure

  1. Initialization
  2. Evaluation
  3. Selection
  4. Crossover and Mutation
  5. Repeat 2-4 until termination



(Online) Black-box Optimization

Genetic Algorithms

- Genetic algorithms are inspired by Darwin's theory of evolution, which explains how species evolve over generations through processes such as selection, crossover, and mutation.

- Procedure

  1. Initialization

     We initialize a population consisting of N solution candidates $\mathbf{X}^C = [x_1^C, \cdots, x_N^C]$

  2. Evaluation

     We evaluate the performance of candidates fitness $\mathbf{f}^C = [f_1^C, \cdots, f_N^C]$

(Online) Black-box Optimization

Genetic Algorithms

- Genetic algorithms are inspired by Darwin's theory of evolution, which explains how species evolve over generations through processes such as selection, crossover, and mutation.

- Procedure

  3. Selection

     Given a set of E parent solutions $\mathbf{X}^P = [x_1^P, \cdots, x_E^P]$, (At first stage, $\mathbf{X}^C = \mathbf{X}^P$),

     We select next generation of parent from selection: $\mathbf{X}^{P'}, \mathbf{f}^{P'} = \text{Selection}(\mathbf{X}^P, \mathbf{f}^P, \mathbf{X}^C, \mathbf{f}^C)$

     Most GA make use of truncation selection (select top-E performing solutions)

(Online) Black-box Optimization

Genetic Algorithms

- Genetic algorithms are inspired by Darwin's theory of evolution, which explains how species evolve over generations through processes such as selection, crossover, and mutation.

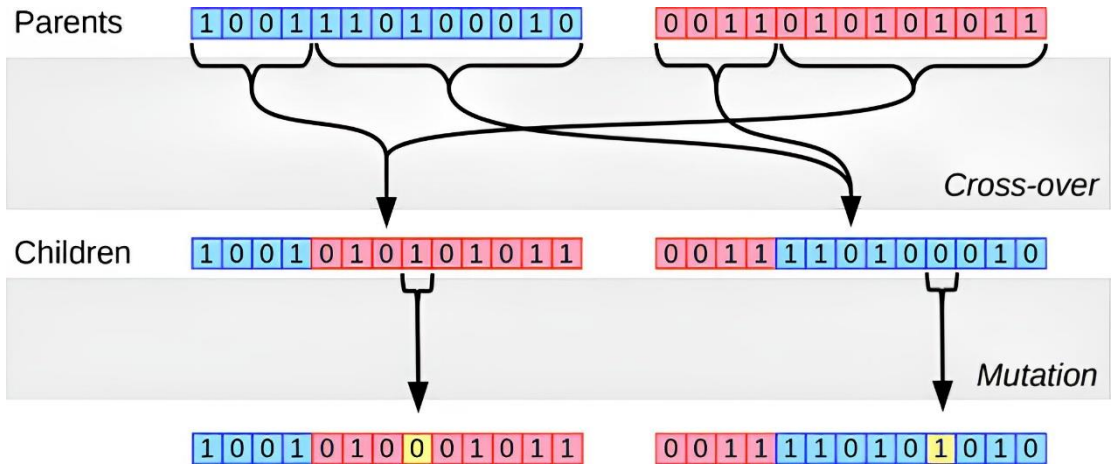
- Procedure

    4. Crossover and Mutation

        Given E parents, we generate N children $\mathbf{X}^C = \left[x_1^C, \cdots, x_N^C\right]$ through crossover and mutation

        Crossover: combine the information of two parents to produce one or more offspring

        Mutation: introduce variation into the offspring by randomly changing parts of its structure



(Online) Black-box Optimization

Evolutionary Algorithms

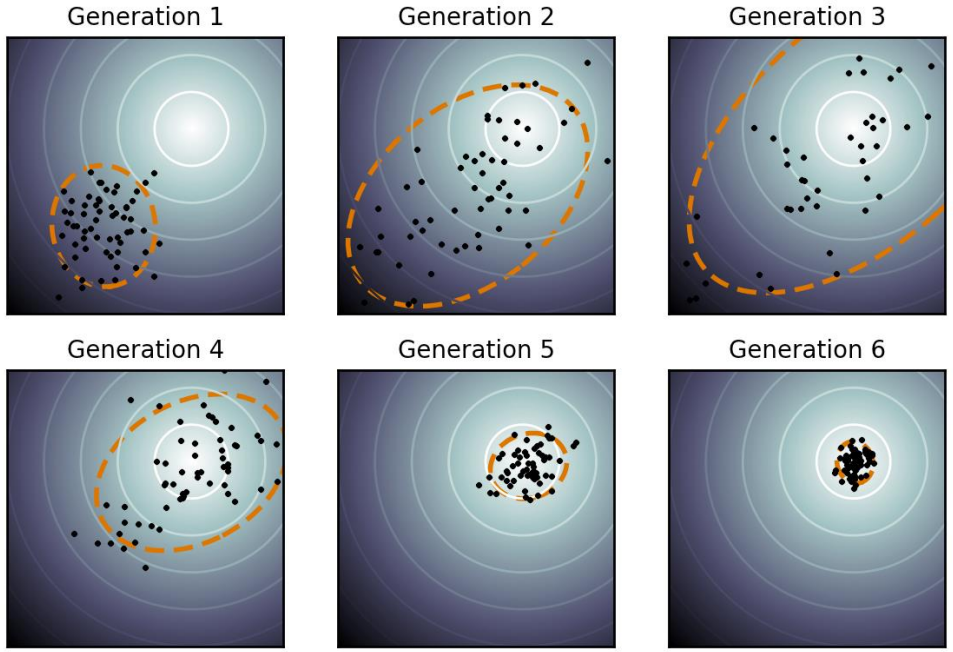
- CMA-ES (Covariance Matrix Adaptation) is a stochastic method for real-parameter (continuous domain) optimization of non-linear, non-convex functions

- It is also widely used in optimizing the acquisition function in Bayesian Optimization

$$x^{(n+1)} = \operatorname*{argmax}_x \mathrm{EI}(x) \triangleq \mathrm{E}[\max\{0, f - f^{\max}\} | \mathcal{D}]$$

- Procedure

  1. Initialization distribution parameters
  2. Generate samples from the distribution and Evaluate
  3. Update distribution parameters



(Online) Black-box Optimization

Evolutionary Algorithms

- CMA-ES (Covariance Matrix Adaptation) is a stochastic method for real-parameter (continuous domain) optimization of non-linear, non-convex functions

- Procedure

  1. Initialization distribution parameters

     In most cases, we use diagonal Gaussian approximations $\mathcal{N}(\cdot\,|\mathbf{m}_0, \boldsymbol{\sigma}_0^2\boldsymbol{I})$

  2. Generate samples from the distribution and Evaluate

     $x_1, x_2, \cdots, x_n \sim \mathcal{N}(\mathbf{m}_t, \boldsymbol{\sigma}_t^2\boldsymbol{I})$ and get fitness $f_1, f_2, \cdots, f_n$

(Online) Black-box Optimization

Evolutionary Algorithms

- CMA-ES (Covariance Matrix Adaptation) is a stochastic method for real-parameter (continuous domain) optimization of non-linear, non-convex functions

- Procedure

3. Update distribution parameters

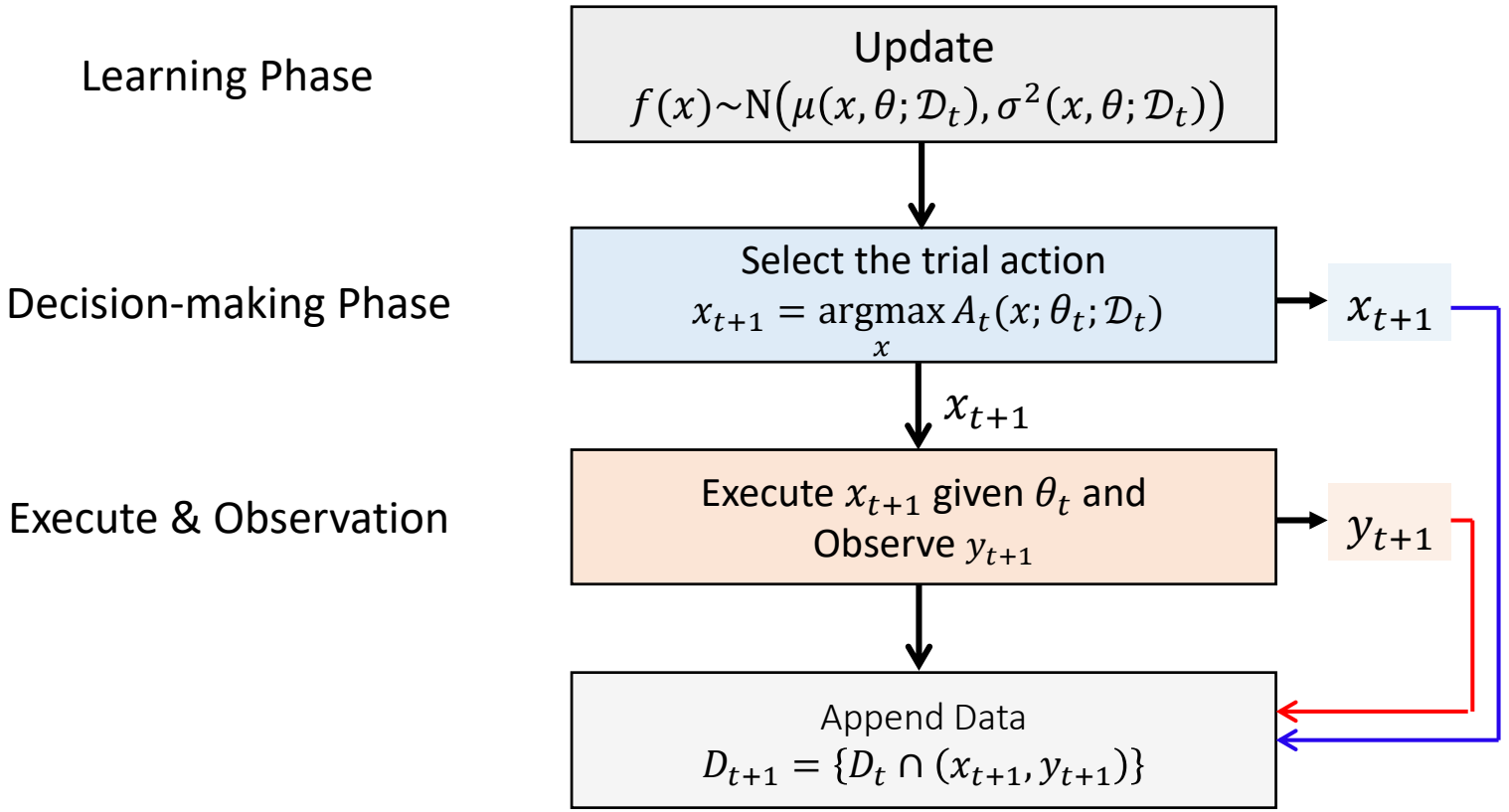
$$\mathbf{m}_{t+1} = (1-\alpha_t)\mathbf{m}_t + a_t \sum_{j=1}^{n} w_{t,j} x_j \,, \qquad \sigma_{t+1} = (1-\alpha_t)\sigma_t + \alpha_t \sqrt{\sum_{j}^{n} w_{t,j} \left(x_j - \mathbf{m}_t\right)^2}$$

$$w_{t,j} = \begin{cases} 1/E & \text{if } rank(j) \leq E \\ 0 & \text{otherwise} \end{cases}$$

(Online) Black-box Optimization

Bayesian Optimization

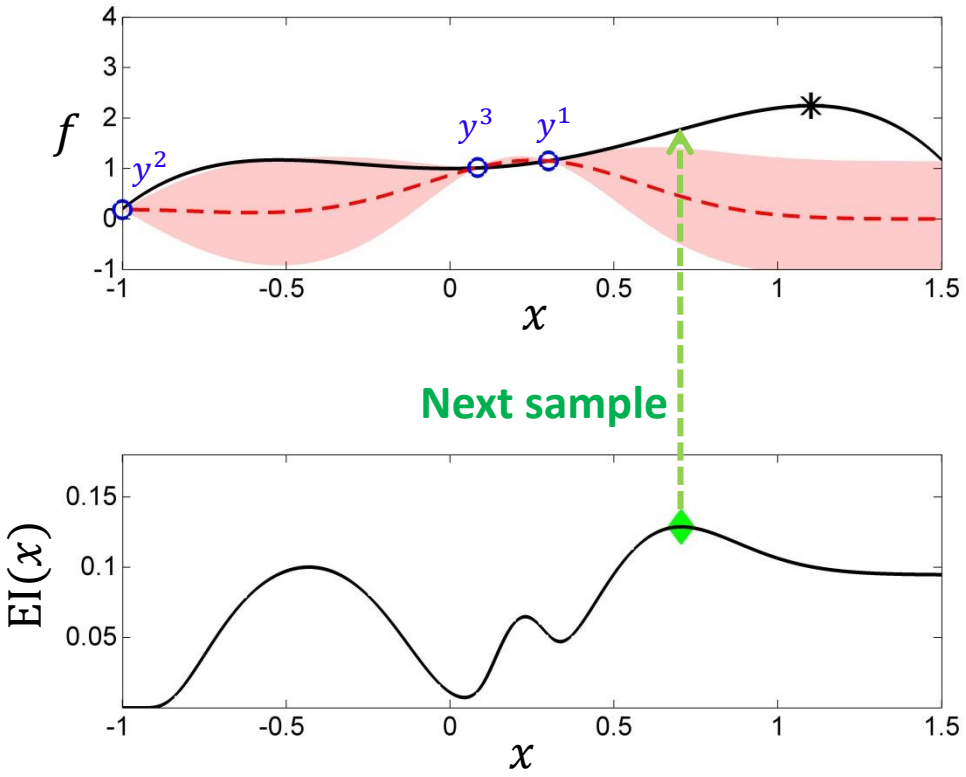
- Bayesian Optimization is one of the most widely used methods to solve black-box optimization problems
- Recap) Bayesian Optimization



(Online) Black-box Optimization

Bayesian Optimization

- Bayesian Optimization is one of the most widely used methods to solve black-box optimization problems
- Recap) Bayesian Optimization



Construct probability distribution on the unknown function value

$$p(f|\mathcal{D}) = \mathrm{N}\big(\mu(x|\mathcal{D}), \sigma^2(x|\mathcal{D})\big)$$

$$\mu(x|\mathcal{D}) = \mathbf{k}^{\mathrm{T}}(\mathbf{K} + \sigma_\epsilon^2\mathbf{I})^{-1}\mathbf{y}_{1:n}$$

$$\sigma^2(x|\mathcal{D}) = k(x,x) - \mathbf{k}^{\mathrm{T}}(\mathbf{K} + \sigma_\epsilon^2\mathbf{I})^{-1}\mathbf{k}$$

Select the next trial action that maximizes

$$x^{(4)} = \underset{x}{\operatorname{argmax}}\,\mathrm{EI}(x) \triangleq \mathbb{E}[\max\{0, f - f^{\max}\}|\mathcal{D}]$$

$$x^{(4)} = 0.70$$

Query the function value

$$y^{(4)} = f(x^{(4)}) + \epsilon, \epsilon \sim \mathrm{N}(0, 0.01^2)$$

$$y^{(4)} = 1.76$$

(Online) Black-box Optimization

Policy Gradient

- Black-box optimization problem can be formulated as

$$\text{find } \mathbf{x}^* = \underset{\mathbf{x}\in\boldsymbol{\mathcal{X}}}{\text{argmax}}\, f(\mathbf{x}) \text{ with a given budget of evaluations } T$$

- Our objective can be defined as discovering policy $\pi_\theta(\mathbf{x})$ such that $\mathbb{E}_{\mathbf{x}\sim\pi(\mathbf{x})}[f(\mathbf{x})]$
- By using policy gradient theorem, we can update policy to maximize the objective by using policy gradient:

$$\nabla_\theta \mathbb{E}_{\mathbf{x}\sim\pi(\mathbf{x})}[f(\mathbf{x})] = \mathbb{E}_{x\sim\pi_\theta(x)}[\nabla_\theta \log \pi_\theta(x) f(x)]$$

(Online) Black-box Optimization

Policy Gradient

- Our objective can be defined as discovering policy $\pi_\theta(\mathbf{x})$ such that $\mathbb{E}_{\mathbf{x}\sim\pi(\mathbf{x})}[f(\mathbf{x})]$
- By using policy gradient theorem, we can update policy to maximize the objective by using policy gradient:

$$\nabla_\theta \mathbb{E}_{\mathbf{x}\sim\pi(\mathbf{x})}[f(\mathbf{x})] = \mathbb{E}_{x\sim\pi_\theta(x)}[\nabla_\theta \log \pi_\theta(x) f(x)]$$

Proof) $$\nabla_\theta \mathbb{E}_{\mathbf{x}\sim\pi_\theta(\mathbf{x})}[f(\mathbf{x})] = \nabla_\theta \int_X \pi_\theta(x) f(x) dx = \nabla_\theta \int_X \pi_\theta(x) f(x) dx$$

$$= \int_X \nabla_\theta \pi_\theta(x) f(x) dx = \int_X \pi_\theta(x) \cdot \nabla_\theta \log \pi_\theta(x)\, f(x) dx \qquad \text{(log-derivative trick)}$$

$$= \mathbb{E}_{x\sim\pi_\theta(x)}[\nabla_\theta \log \pi_\theta(x) f(x)]$$

(Online) Black-box Optimization

Policy Gradient

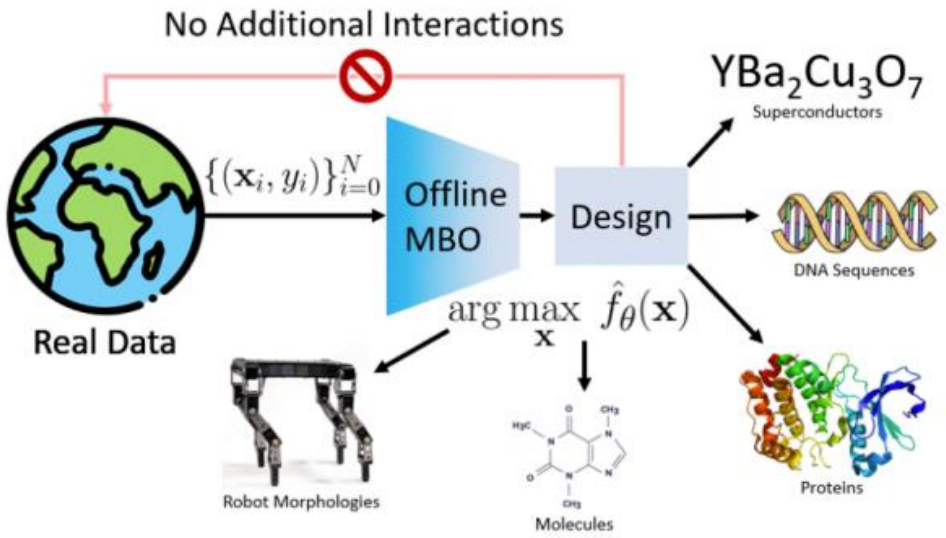
- Our objective can be defined as discovering policy $\pi_\theta(\mathbf{x})$ such that $\mathbb{E}_{\mathbf{x}\sim\pi(\mathbf{x})}[f(\mathbf{x})]$
- By using policy gradient theorem, we can update policy to maximize the objective by using policy gradient:

$$\nabla_\theta \mathbb{E}_{\mathbf{x}\sim\pi(\mathbf{x})}[f(\mathbf{x})] = \mathbb{E}_{x\sim\pi_\theta(x)}[\nabla_\theta \log \pi_\theta(x) f(x)]$$

[Policy Gradient for Black-box Optimization]

Initialize $\pi_{\theta_0}(\cdot)$
For round $r = 1, \cdots, R-1$ do
  Sample $x_1^r, x_2^r, \cdots x_n^r \sim \pi_\theta(\cdot)$ and evaluate $y_1^r, y_2^r, \cdots, y_n^r = f(x_1^r), f(x_2^r), \cdots, f(x_n^r)$
  Update θ using the policy gradient: $\theta_{r+1} \leftarrow \theta_r + \frac{1}{n}\sum_{i=1}^{n} \nabla_\theta \log \pi_{\theta_r}(x_i^r) \cdot f(x_i^r)$

Motivation



- In real world optimization problems, evaluation of objective function is expensive and prohibitive
  Examples) Molecule Design, Automated Aircraft Design, …

- Therefore, constructing a data-driven model given a fixed, offline dataset is crucial
- Formally, we try to find an input $\boldsymbol{x}$ such that

$$\text{find } \mathbf{x}^* = \underset{\mathbf{x}}{\text{argmax}}\, f(\mathbf{x}) \text{ with } \textbf{only} \text{ a fixed dataset, } D = \{(\mathbf{x}_1, f(\mathbf{x}_1)), \ldots, (\mathbf{x}_N, f(\mathbf{x}_N))\}$$

Motivation

- Formally, we try to find an input ***x*** such that

$$\text{find } \mathbf{x}^* = \underset{\mathbf{x}}{\text{argmax}}\, f(\mathbf{x}) \text{ with } \textbf{only} \text{ a fixed dataset, } D = \left\{\left(\mathbf{x}_1, f(\mathbf{x}_1)\right), \ldots, \left(\mathbf{x}_N, f(\mathbf{x}_N)\right)\right\}$$

- Main Approaches

  1. Surrogate model-based: Approximate $f(x)$ using previously collected data, choose the query that maximizes the surrogate model

  2. Generative model-based: Learn an inverse function $f^{-1}(y)$ using previously collected data, choose the query generated from the inverse function given by a desired output.

- Today, we'll focus on Surrogate model-based approaches

Prior Works – Naïve Approach

- A straightforward way to solve Design Optimization is by estimating a ground-truth function with a feed-forward model using a supervised learning framework and performing gradient ascent to find the optimal point

$$\textbf{Step 1}.\ \boldsymbol{\theta}^* = \underset{\theta}{\operatorname{argmin}} \frac{1}{N} \sum_{i=1}^{N} \left(f_\theta(\mathbf{x}_i) - f(\mathbf{x}_i) \right)^2 \text{ where } D = \left\{ \left(\mathbf{x}_i, f(\mathbf{x}_i) \right) \right\}_{i=1}^{N}$$

$$\textbf{Step 2}.\ \text{choose } \mathbf{x}^* = \underset{\mathbf{x}}{\operatorname{argmax}}\, f_{\theta^*}(\mathbf{x})$$

Prior Works – Naïve Approach

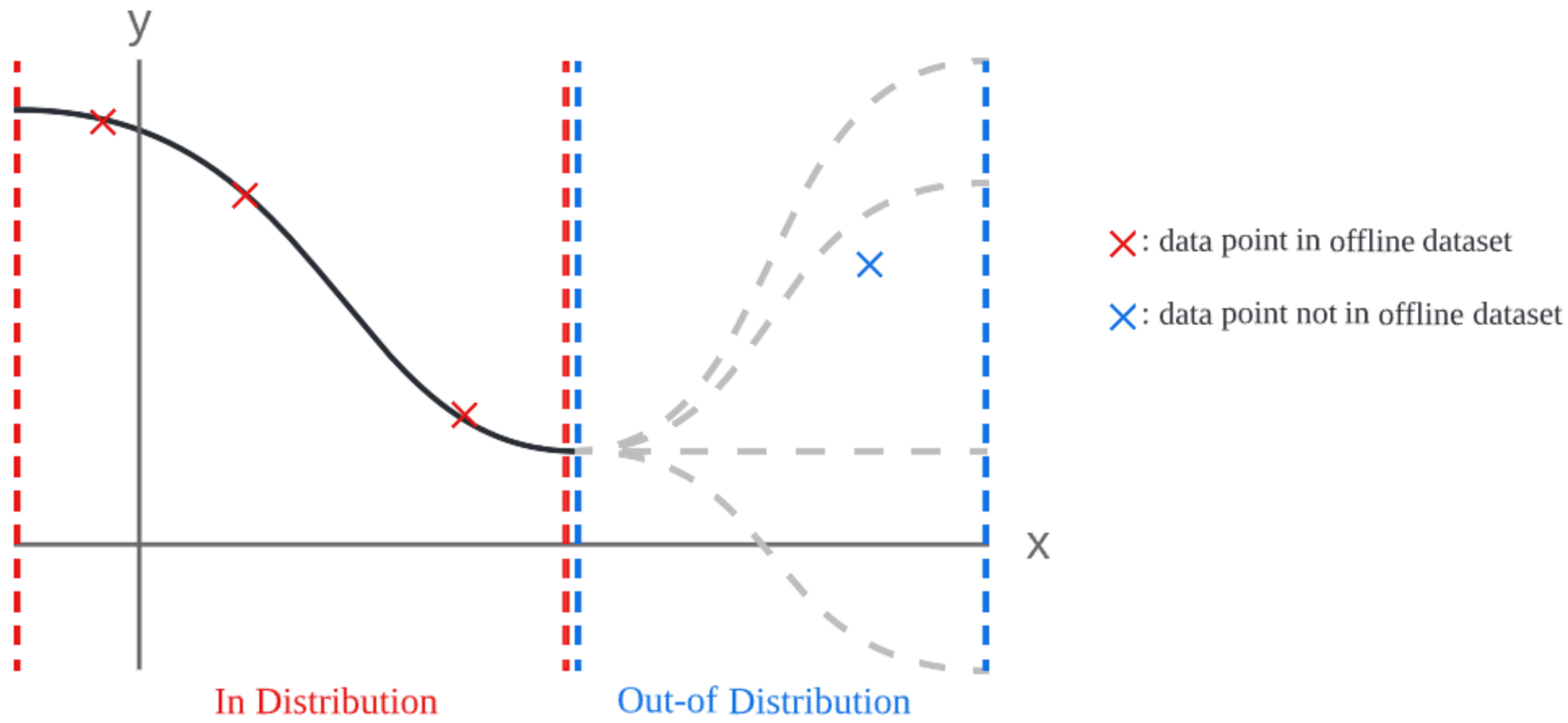
- A straightforward way to solve Design Optimization is by estimating a ground-truth function with a feed-forward model using a supervised learning framework and performing gradient ascent to find the optimal point

$$\textbf{Step 1. } \boldsymbol{\theta}^* = \underset{\theta}{\operatorname{argmin}} \frac{1}{N} \sum_{i=1}^{N} \left(f_\theta(\mathbf{x}_i) - f(\mathbf{x}_i) \right)^2 \text{ where } D = \left\{ \left(\mathbf{x}_i, f(\mathbf{x}_i) \right) \right\}_{i=1}^{N}$$

$$\textbf{Step 2. } \text{choose } \mathbf{x}^* = \underset{\mathbf{x}}{\operatorname{argmax}}\, f_{\theta^*}(\mathbf{x})$$

Problem 1. Trained model is only valid near the training distribution, which leads to a large error in out-of-distribution input values. However, we must extrapolate from the training distribution!!

Prior Works – Naïve Approach



Prior Works – Naïve Approach

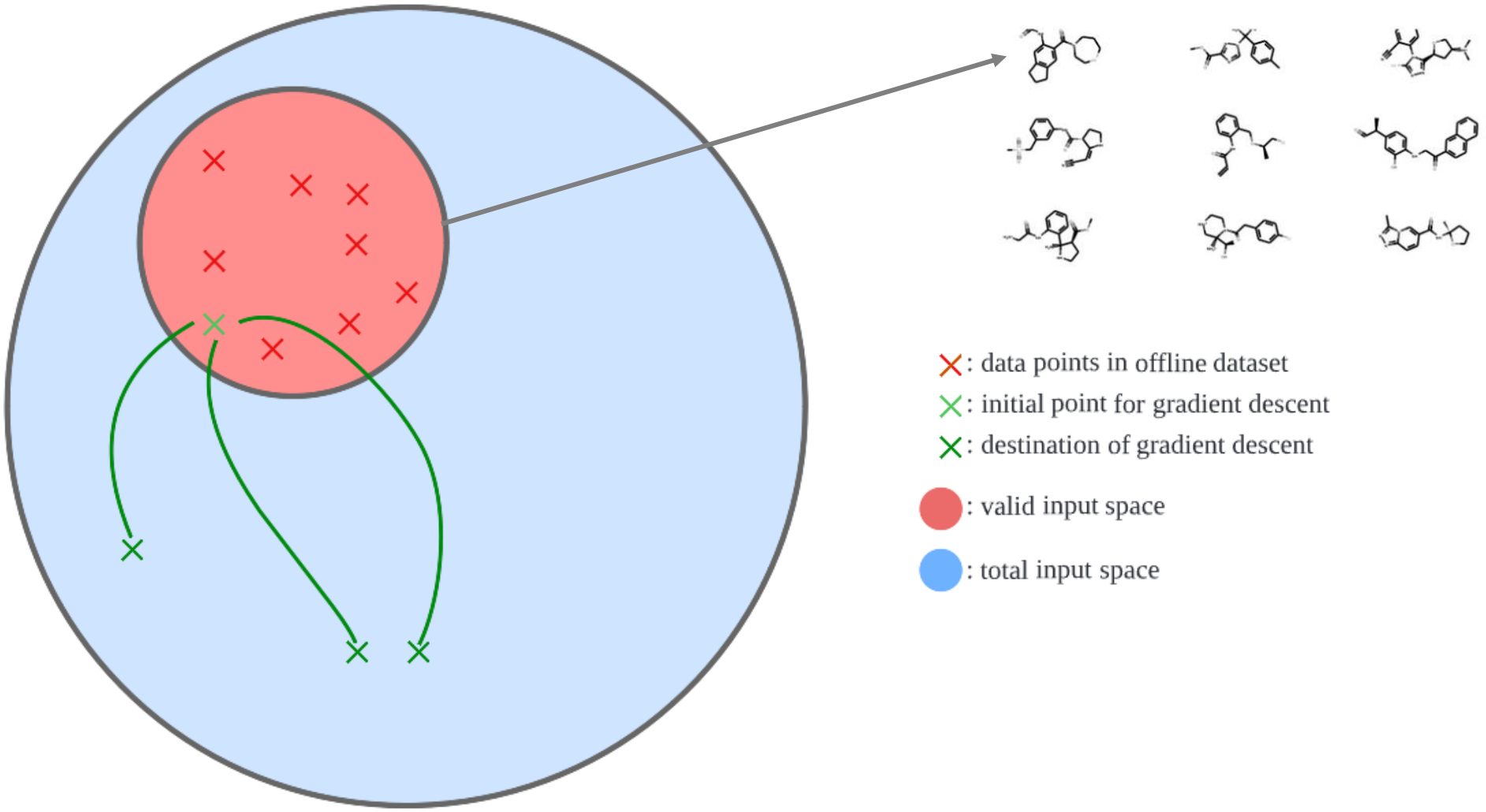
- A straightforward way to solve Design Optimization is by estimating a ground-truth function with a feed-forward model using a supervised learning framework and performing gradient ascent to find the optimal point

$$\textbf{Step 1.}\ \boldsymbol{\theta}^* = \operatorname*{argmin}_{\theta} \frac{1}{N} \sum_{i=1}^{N} \left(f_\theta(\mathbf{x}_i) - f(\mathbf{x}_i) \right)^2 \ \text{where}\ D = \left\{ \left(\mathbf{x}_i, f(\mathbf{x}_i) \right) \right\}_{i=1}^{N}$$

$$\textbf{Step 2.}\ \text{choose}\ \mathbf{x}^* = \operatorname*{argmax}_{\mathbf{x}} f_{\theta^*}(\mathbf{x})$$

Problem 2. Searching an input that maximizes a proxy model can be done by gradient ascent. However, since only a small manifold of input space is valid, gradient ascent can easily fall into invalid input especially in high-dimensional space.

Prior Works – Naïve Approach



Key Challenges

Design Optimization algorithms are able to …

1. Reason about uncertainty and out-of-distribution input values

2. Search possible input values while staying on the manifold of valid input space

How can we cope with such challenges?